Listen

Lab 11: TreeType Ancestors

50 points · Unit 6 · Due Sunday, November 1, 11:59 PM

> This lab has a **Mini Oral Inquiry** (Mini Oral 5). The tree in Figure 1 is a valid binary search tree - verified - and the ancestors of T resolve to exactly the expected output `J P U S`.

A single recursive function, and it is harder than it looks.

The insight is that the search path from the root to a node *is* the ancestor list. You do not need to search the whole tree — the binary search tree property tells you which way to go at every step, and every node you pass through is an ancestor.

The ordering requirement is where most submissions go wrong. You discover ancestors on the way down, from the root toward the node, but the required output reads root-first — so think carefully about whether you are appending on the way down or on the way back up.

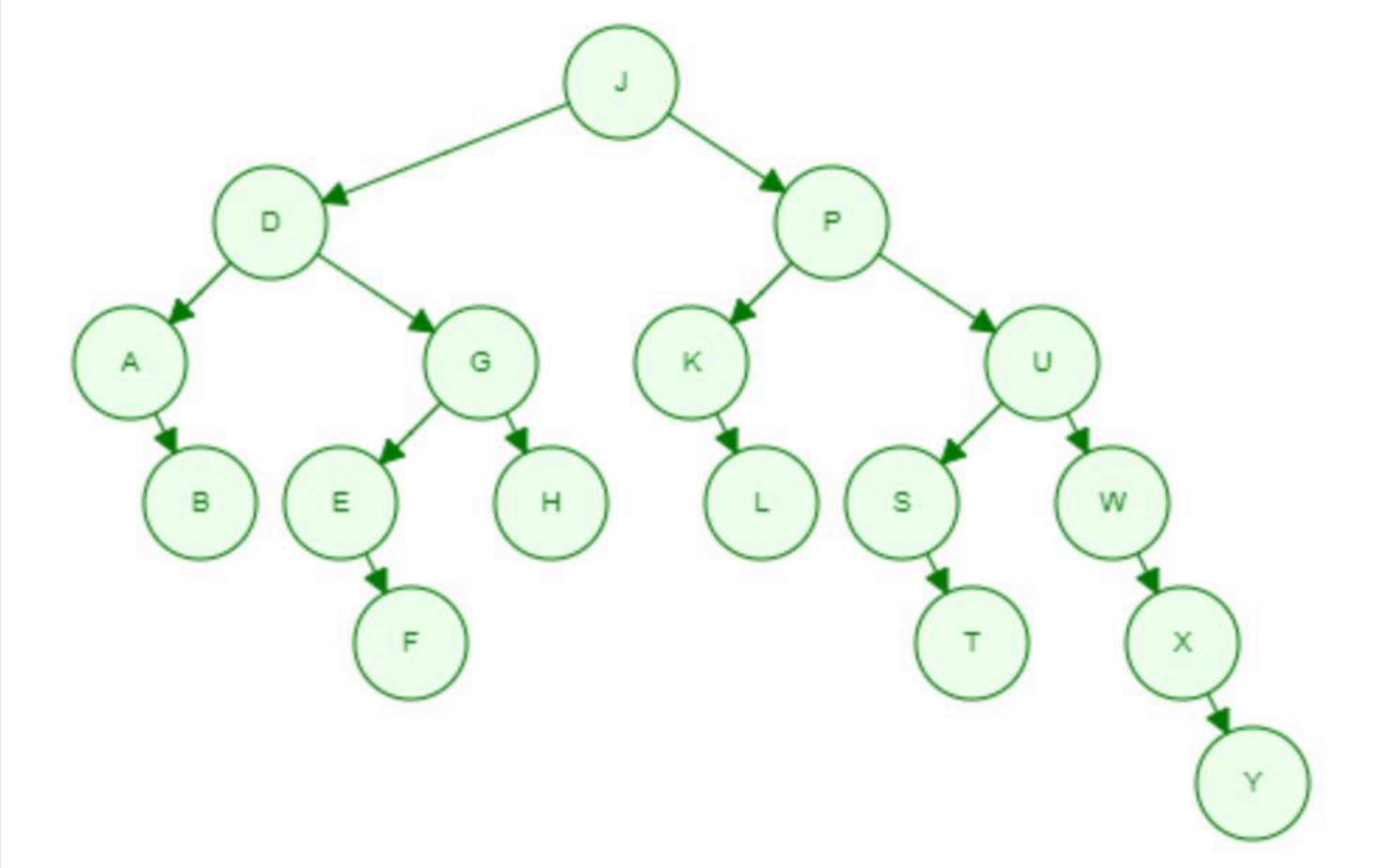
Specification

Write a **recursive** `TreeType` member function called `Ancestors` that returns a string representing the ancestors of the node whose `info` member contains `value`.

Use this signature:

```
string TreeType<ItemType>::Ancestors(ItemType item) const
```

Figure 1 — the test tree.



For the tree above, the ancestors of **T** are S, U, P, and J — the nodes you pass through walking from T back up to the root.

Test your function with the provided driver and verify the expected output:

```
J P U S
```

Note the relationship between those two statements: the ancestors listed walking *up* from T are S, U, P, J, and the required output is that sequence reversed — root first, immediate parent last.

Files Provided

Download `Lab 11 Files` — it is the item directly below this document in Unit 6. Everything listed here is in that one zip. Unzip it into a single folder and compile there.

- `Ancestors_TreeType.zip` — the templated binary search tree with its implementation and a driver, plus the `QueType` the tree algorithms use. **This is what the lab extends.**
- `CSC240_BST.zip` — supplemental. A BST class with a sample driver, for reference and examples.

Creative Final Feature (20 points)

This is not extra credit and it is not optional. **Twenty of this lab's 50 points** come from adding something the specification did not ask for. Eight of the fifteen labs carry a Creative Final Feature — one per unit — and this is one of them.

Extend the traversal. Directions: a `Depth(item)` returning the node's depth, a `Path(item)` including the node itself, a `CommonAncestor(a, b)` returning the lowest common ancestor, an iterative version for comparison, or a `Descendants(item)`.

> **Feature Documentation** — answer all four, in a comment block at the top of your main file:
>
> 1. What does your feature do?
> 2. What concept from your AI Learning Query did it use?
> 3. What did you try that did not work?
> 4. What would you do next if you had another week?

AI Learning Query

Before you write your Creative Final Feature, have this conversation. Start with the prompt below, continue for at least five exchanges, paste your own code in at least once, and push back at least once when an answer seems wrong or incomplete. **This is the graded transcript for Unit 6** — submit it to the AI Learning Transcript 6 dropbox.

> *"I need to find all ancestors of a node in a binary search tree in C++ and return them as a string, root first. Do not write the function. Help me think about it: what does the search path from the root have to do with the ancestor set, and should I build the string on the way down the recursion or on the way back up? Explain what changes about the resulting order in each case, and what should happen when the value is not in the tree at all."*

STL Counterpart

> **Ungraded — read this, do not submit it.** `std::set` and `std::map` are balanced binary search trees, but they deliberately expose no tree structure — no way to ask for a node's parent or depth. That is a design decision worth thinking about: the standard library hides the tree because clients should depend on the ordering, not the shape. Your lab requires the shape. Your submission implements the traversal yourself.

Be Ready to Answer

These are the questions for **Mini Oral Inquiry 5**. You will record yourself answering them with your code on screen. Question 1 is also submitted in writing as part of this lab.

Every question below is about *your* submission. You may use AI to prepare — that is encouraged — but the answers must describe the code you actually wrote.

1. State your base case or cases, and the invariant your recursion maintains about the string built so far. Write this as a comment above the function.
2. Explain how *your* solution produces root-first ordering. Did you append on the way down or on the way back up? Show the line.
3. What is the running time of your `Ancestors`? Answer in terms of the tree's height, then say what that becomes for a balanced tree and for a tree built from sorted input.
4. Call `Ancestors` on a value that is not in the tree. Then call it on the root itself. Show both outputs and say whether each is what you intended.
5. Your function relies on the binary search tree property to choose a direction at each node. What would you have to change if the tree were an ordinary binary tree with no ordering?

Grading

Criterion	What earns the points	Pts
Correctness and Specification	Every requirement in the specification is met and demonstrated by the driver. Output matches what is stated above. Edge cases named in the specification behave correctly.	**15**
Data Structure Implementation	The required structure is implemented as specified. No STL container is substituted for work the specification asks you to do yourself. Memory is allocated and released correctly.	**8**
Design and Documentation	Interface separated from implementation. Preconditions and postconditions documented on every public function you write or change.	**4**
Coding Conventions	Follows the Oakton C++ Coding Conventions: naming, indentation, commenting, file organization.	**3**

Required work subtotal	**30**
CFF — Feature implemented and runs without errors	**6**
CFF — Feature is genuinely creative and self-chosen	**4**
CFF — Feature demonstrates the concept from your AI Learning Query	**6**
CFF — Feature Documentation complete (all four questions)	**4**
Total	**50**

Submitting. Upload all source files plus your driver to the Lab 11 dropbox. **Late submissions are not accepted** — the dropbox closes at the due date. Office hours are virtual, 30-minute slots, Monday through Friday 7:00–8:00 PM; the booking link is in *Start Here*.